目次

章	シート名	内容
1	変更履歴	改訂履歴
2	01_概要	システム概要/目的/範囲
3	02_構成	アーキテクチャ/構成要素
4	03_クラス一覧	全クラス一覧
5	04_Web詳細	画面系Servlet詳細
6	05_API詳細	API詳細
7	06_DAO詳細	DAO仕様
8	07_モデル詳細	ドメインモデル詳細
9	08_DTO詳細	DTO詳細
10	09_ユーティリティ	ユーティリティ詳細
11	10_DB詳細	テーブル/カラム詳細
12	11_認可	認証/認可ルール
13	12_エラーハンドリング	例外/エラー応答
14	13_ビルド実行	ビルド/実行手順
15	14_処理フロー	Servlet処理フロー
16	15_SQL詳細	SQL詳細
17	16_シーケンス	主要シーケンス図

備考

変更履歴

版	日付	担当	内容
1.2	2026-02-08	Codex	API分岐/認可/備考の精度改善

システム概要

項目
システム名
目的
対象ユーザー
提供機能
技術スタック

内容
プロジェクト管理システム
WBS/**ガント/工数/予算/コミュニケーションを統合し、プロジェクト計画から実行までを一元管理する。**
ADMIN / MANAGER / MEMBER / VIEWER
プロジェクト管理、計画編集、かんばん、カレンダー、レポート、予実管理、フォーラム/チャット、ドキュメント、課題管理、承認ワークフロー、Wiki**、ユーザー管理**
Java 17, Jakarta Servlet 5.0, JSP/JSTL, MySQL 8, Jackson, Maven

アーキテクチャ構成

層	主な構成要素
Web	Servlet/JSP/Filter
API	JSON API Servlet/Filter
DAO	JDBC DAO
Model/DTO	**ドメイン/転送オブジェクト**
Util	Config/DbUtil/HashUtil/WebUtil
DB	MySQL 8

説明
画面表示・ページ遷移・認証/認可。
SPAフロント向けJSON API。
MySQLアクセス。
画面/APIとDBのデータ構造を表現。
共通処理。
プロジェクト管理の永続化。

クラス一覧

分類	パッケージ	クラス
Util	jp.co.example.pm.util	DbUtil
Util	jp.co.example.pm.util	HashUtil
Util	jp.co.example.pm.util	Config
Web Servlet	jp.co.example.pm.web	IndexServlet
Other	jp.co.example.pm.web	WebUtil
Web Servlet	jp.co.example.pm.web	DocsServlet
Web Servlet	jp.co.example.pm.web	LogoutServlet
Web Servlet	jp.co.example.pm.web	ProjectsServlet
Web Servlet	jp.co.example.pm.web	CalendarServlet
Filter	jp.co.example.pm.web	EncodingFilter
Web Servlet	jp.co.example.pm.web	ReportsServlet
Web Servlet	jp.co.example.pm.web	FinanceServlet
Other	jp.co.example.pm.web	AccessControl
Web Servlet	jp.co.example.pm.web	IssuesServlet
Web Servlet	jp.co.example.pm.web	WikiServlet
Web Servlet	jp.co.example.pm.web	LoginServlet
Web Servlet	jp.co.example.pm.web	ProjectServlet
Web Servlet	jp.co.example.pm.web	WorkflowServlet
Web Servlet	jp.co.example.pm.web	KanbanServlet
Other	jp.co.example.pm.web	SessionKeys
Web Servlet	jp.co.example.pm.web	AdminUsersServlet
Filter	jp.co.example.pm.web	AuthFilter
Web Servlet	jp.co.example.pm.web	ForumServlet
DTO	jp.co.example.pm.web.dto	UserRequest
DTO	jp.co.example.pm.web.dto	UserSummary
DTO	jp.co.example.pm.web.dto	ProjectPlan
Other	jp.co.example.pm.web.api	ApiErrorResponse
API Servlet	jp.co.example.pm.web.api	ApiChatServlet
API Servlet	jp.co.example.pm.web.api	ApiProjectsServlet
Filter	jp.co.example.pm.web.api	ApiExceptionFilter
API Servlet	jp.co.example.pm.web.api	ApiUsersServlet
API Servlet	jp.co.example.pm.web.api	ApiReportsServlet
API Servlet	jp.co.example.pm.web.api	ApiForumServlet
API Servlet	jp.co.example.pm.web.api	ApiFinanceServlet
API Servlet	jp.co.example.pm.web.api	ApiAlertsServlet
API Servlet	jp.co.example.pm.web.api	ApiIssuesServlet
API Servlet	jp.co.example.pm.web.api	ApiWikiServlet
API Servlet	jp.co.example.pm.web.api	ApiWorkflowServlet
API Servlet	jp.co.example.pm.web.api	ApiPlanServlet
API Servlet	jp.co.example.pm.web.api	ApiBaselineServlet
API Servlet	jp.co.example.pm.web.api	ApiDocumentsServlet
API Servlet	jp.co.example.pm.web.api	ApiTasksServlet
DAO	jp.co.example.pm.dao	ProjectMemberDao

DAO	jp.co.example.pm.dao	DocumentDao
DAO	jp.co.example.pm.dao	UserDao
DAO	jp.co.example.pm.dao	IssueDao
DAO	jp.co.example.pm.dao	ForumPostDao
DAO	jp.co.example.pm.dao	ProjectFinanceDao
DAO	jp.co.example.pm.dao	TaskDao
DAO	jp.co.example.pm.dao	ForumThreadDao
DAO	jp.co.example.pm.dao	AssignmentDao
DAO	jp.co.example.pm.dao	ChatDao
DAO	jp.co.example.pm.dao	ResourceDao
DAO	jp.co.example.pm.dao	WikiDao
DAO	jp.co.example.pm.dao	WorkflowDao
DAO	jp.co.example.pm.dao	ProjectDao
DAO	jp.co.example.pm.dao	DependencyDao
Model	jp.co.example.pm.model	Team
Model	jp.co.example.pm.model	ProjectFinance
Model	jp.co.example.pm.model	ForumThread
Model	jp.co.example.pm.model	DocumentFile
Model	jp.co.example.pm.model	Resource
Model	jp.co.example.pm.model	ForumPost
Model	jp.co.example.pm.model	User
Model	jp.co.example.pm.model	TaskAssignment
Model	jp.co.example.pm.model	ProjectMember
Model	jp.co.example.pm.model	Issue
Model	jp.co.example.pm.model	WikiPage
Model	jp.co.example.pm.model	Task
Model	jp.co.example.pm.model	Project
Model	jp.co.example.pm.model	WorkflowRequest
Model	jp.co.example.pm.model	ChatMessage
Model	jp.co.example.pm.model	Dependency

役割
DbUtil: 共通ユーティリティ。
HashUtil: 文字列のハッシュ生成を行う。
Config: 共通ユーティリティ。
IndexServlet: ルートアクセスの誘導。
WebUtil: Web層の共通部品。
DocsServlet: ドキュメント画面を表示する。
LogoutServlet: ログアウト処理を行う。
ProjectsServlet: プロジェクト一覧画面を表示する。
CalendarServlet: カレンダー画面を表示する。
EncodingFilter: 文字コードをUTF-8に固定する。
ReportsServlet: レポート画面を表示する。
FinanceServlet: 予実管理画面を表示する。
AccessControl: 認可チェックを行う。
IssuesServlet: 不具合管理画面を表示する。
WikiServlet: Wiki画面を表示する。
LoginServlet: ログイン画面と認証処理を行う。
ProjectServlet: プロジェクト詳細画面を表示する。
WorkflowServlet: ワークフロー画面を表示する。
KanbanServlet: かんばん画面を表示する。
SessionKeys: セッションキー定数を保持する。
AdminUsersServlet: ユーザー管理画面を表示する。
AuthFilter: 認証チェックを行う。
ForumServlet: フォーラム画面を表示する。
UserRequest: ユーザーAPIの入力DTO。
UserSummary: 公開用ユーザーサマリ。
ProjectPlan: プロジェクト計画の集約DTO。
ApiErrorResponse: APIエラーのレスポンスDTO。
ApiChatServlet: チャットAPI。
ApiProjectsServlet: プロジェクトのCRUD API。
ApiExceptionFilter: APIの例外をJSONで返す。
ApiUsersServlet: ユーザー管理API。
ApiReportsServlet: レポート情報API。
ApiForumServlet: フォーラムAPI。
ApiFinanceServlet: プロジェクト予実管理API。
ApiAlertsServlet: アラート情報API。
ApiIssuesServlet: 不具合管理API。
ApiWikiServlet: Wiki管理API。
ApiWorkflowServlet: 承認ワークフローAPI。
ApiPlanServlet: プロジェクト計画の取得/保存。
ApiBaselineServlet: ベースライン保存を行う。
ApiDocumentsServlet: ドキュメントAPI。
ApiTasksServlet: タスク管理API。
ProjectMemberDao: プロジェクト参加者のDBアクセス。

DocumentDao: ドキュメント情報のDBアクセス。
UserDao: ユーザー情報のDBアクセス。
IssueDao: 不具合情報のDBアクセス。
ForumPostDao: フォーラム投稿のDBアクセス。
ProjectFinanceDao: プロジェクト予実のDBアクセス。
TaskDao: タスク情報のDBアクセス。
ForumThreadDao: フォーラムスレッドのDBアクセス。
AssignmentDao: タスク割当のDBアクセス。
ChatDao: チャット情報のDBアクセス。
ResourceDao: リソース情報のDBアクセス。
WikiDao: Wikiページのの DBアクセス。
WorkflowDao: 承認ワークフローのDBアクセス。
ProjectDao: プロジェクト情報のDBアクセス。
DependencyDao: 依存関係のDBアクセス。
Team: チーム情報を保持する。
ProjectFinance: プロジェクト予実情報を保持する。
ForumThread: フォーラムのスレッド情報を保持する。
DocumentFile: ドキュメント情報を保持する。
Resource: リソース情報を保持する。
ForumPost: フォーラムの投稿情報を保持する。
User: ユーザー情報を保持する。
TaskAssignment: タスクとリソースの割当を保持する。
ProjectMember: プロジェクト参加者情報を保持する。
Issue: 不具合情報を保持する。
WikiPage: Wikiページ情報を保持する。
Task: タスク/WBS情報を保持する。
Project: プロジェクト情報を保持する。
WorkflowRequest: 承認ワークフロー情報を保持する。
ChatMessage: チャットメッセージ情報を保持する。
Dependency: タスク依存関係を保持する。

Web Servlet 詳細

クラス	URL	HTTP	JSP/遷移
IndexServlet	/	GET	Redirect
DocsServlet	/docs	GET	docs.jsp
LogoutServlet	/logout	GET	Redirect
ProjectsServlet	/projects	GET	projects.jsp
CalendarServlet	/calendar	GET	calendar.jsp
ReportsServlet	/reports	GET	reports.jsp
FinanceServlet	/finance	GET	finance.jsp
IssuesServlet	/issues	GET	issues.jsp
WikiServlet	/wiki	GET	wiki.jsp
LoginServlet	/login	GET	login.jsp
LoginServlet	/login	POST	Redirect
ProjectServlet	/project	GET	project.jsp
WorkflowServlet	/workflow	GET	workflow.jsp
KanbanServlet	/kanban	GET	kanban.jsp
AdminUsersServlet	/admin/users	GET	admin_users.jsp
ForumServlet	/forum	GET	forum.jsp

認可	入力
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
なし	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
なし	
なし	Params: loginId, password
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter)	
ログイン必須 (AuthFilter) / ADMIN	
ログイン必須 (AuthFilter)	

出力/備考
Redirect
JSP: docs.jsp
Redirect
JSP: projects.jsp
JSP: calendar.jsp
JSP: reports.jsp
JSP: finance.jsp
JSP: issues.jsp
JSP: wiki.jsp
JSP: login.jsp
Redirect
JSP: project.jsp
JSP: workflow.jsp
JSP: kanban.jsp
JSP: admin_users.jsp
JSP: forum.jsp

API 詳細

クラス	Path	HTTP	認可
ApiChatServlet	/api/chat	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiChatServlet	/api/chat	POST	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiProjectsServlet	/api/projects/{id}	GET	ログイン必須 (AuthFilter) / LOGIN,PROJECT_MEMBER
ApiProjectsServlet	/api/projects/*	POST	ログイン必須 (AuthFilter) / MANAGER
ApiProjectsServlet	/api/projects/{id}	PUT	ログイン必須 (AuthFilter) / MANAGER,PROJECT_MEMBER
ApiProjectsServlet	/api/projects/{id}	DELETE	ログイン必須 (AuthFilter) / ADMIN
ApiUsersServlet	/api/users/*	GET	ログイン必須 (AuthFilter)
ApiUsersServlet	/api/users/*	POST	ログイン必須 (AuthFilter) / ADMIN
ApiUsersServlet	/api/users/{id}	PUT	ログイン必須 (AuthFilter) / ADMIN
ApiUsersServlet	/api/users/{id}	DELETE	ログイン必須 (AuthFilter) / ADMIN
ApiReportsServlet	/api/reports	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiForumServlet	/api/forum/*	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiForumServlet	/api/forum/*	POST	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiFinanceServlet	/api/finance	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiFinanceServlet	/api/finance	PUT	ログイン必須 (AuthFilter) / MANAGER,PROJECT_MEMBER
ApiAlertsServlet	/api/alerts	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiIssuesServlet	/api/issues/*	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiIssuesServlet	/api/issues/*	POST	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiIssuesServlet	/api/issues/{id}	PUT	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiIssuesServlet	/api/issues/{id}	DELETE	ログイン必須 (AuthFilter) / PROJECT_MEMBER
ApiWikiServlet	/api/wiki/*	GET	ログイン必須 (AuthFilter) / PROJECT_MEMBER

ApiWikiServlet	/api/wiki/*	POST	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiWorkflowServlet	/api/workflow/*	GET	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiWorkflowServlet	/api/workflow/*	POST	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiWorkflowServlet	/api/workflow/{id}	PUT	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiPlanServlet	/api/plan	GET	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiPlanServlet	/api/plan	PUT	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiBaselineServlet	/api/plan/baseline	POST	**ログイン必須** (AuthFilter)
ApiDocumentsServlet	/api/docs	GET	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiDocumentsServlet	/api/docs	POST	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiDocumentsServlet	/api/docs	DELETE	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiTasksServlet	/api/tasks/*	GET	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiTasksServlet	/api/tasks/*	POST	**ログイン必須** (AuthFilter) / PROJECT_MEMBER
ApiTasksServlet	/api/tasks/{id}	PUT	**ログイン必須** (AuthFilter) / PROJECT_MEMBER

入力	出力	Status
Query: projectId	200:list	
Query: projectId Body: ChatMessage		201
Path: {id}	200:projects 200:projects 200:project	404
Body: Project	201:created	
Path: {id} Body: Project	200:updated	400
Path: {id}		204 400
	200:users 200:summaries	
Body: UserRequest	201:created	
Path: {id} Body: UserRequest	200:existing	400 404
Path: {id}		204 400
Query: projectId	200:out	
Query: projectId, threadId	200:posts 200:threads	404
Query: projectId, threadId Body: ForumPost, ForumThread	201:thread	201 404
Query: projectId	200:out	
Query: projectId Body: Map		204 404
Query: projectId	200:alerts	
Query: projectId	200:list	
Query: projectId Body: Issue	201:input	
Path: {id} Body: Issue	200:input	400 404
Path: {id}		204 400 404
Query: id, projectId	200:page 200:list	404

Query: projectId Body: WikiPage	201:input	
Query: projectId	200:list	
Query: projectId Body: WorkflowRequest	201:input	
Path: {id} Body: WorkflowRequest	200:input	400 404
Query: projectId	200:plan	404
Query: projectId Body: ProjectPlan		204
Query: projectId		204
Query: id, projectId	200:list 200: file download	404
Query: projectId, taskId Multipart: file	201:doc	
Query: id		204 400 404
Query: projectId	200:tasks	
Query: projectId Body: Task	201:input	
Path: {id} Body: Task	200:input	400

Validation/Errors	備考
projectId は必須です。 projectId が不正です。	
メッセージは必須です。 projectId は必須です。 projectId が不正です。	
不正なIDです。	ADMIN: 全件 / 非ADMIN: 所属プロジェクトのみ / Path未指定:一覧 / Path指定:{id}:詳細
プロジェクト名は必須です。	
プロジェクト名は必須です。 不正なIDです。	Path必須:{id} (未指定は400)
不正なIDです。	Path必須:{id} (未指定は400)
	ADMIN: 全件(User) / 非ADMIN: 有効ユーザーのみ(UserSummary)
ログインIDは必須です。 氏名は必須です。 パスワードは必須です。	
不正なIDです。	Path必須:{id} (未指定は400)
不正なIDです。	Path必須:{id} (未指定は400)
projectId は必須です。 projectId が不正です。	
	Path=/posts: threadId必須(投稿) / Path未指定: projectId必須(スレッド) / /posts: 200 posts / base: 200 threads
	Path=/posts: threadId必須(投稿) / Path未指定: projectId必須(スレッド) / /posts: 201のみ (Bodyなし)
projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
タイトルは必須です。 projectId は必須です。 projectId が不正です。	
不正なIDです。	Path必須:{id} (未指定は400)
不正なIDです。	Path必須:{id} (未指定は400)
projectId は必須です。 projectId が不正です。	

タイトルは必須です。 projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
不正なIDです。	Path必須:{id} (未指定は400)
projectId は必須です。 projectId が不正です。	
計画データがありません。 プロジェクト名は必須です。 リソース名は必須です。 projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	id指定時: ファイルDL (Content-Disposition/Content-Type) / projectId 指定時: JSON一覧
ファイルがありません。 projectId は必須です。 projectId が不正です。	
projectId は必須です。 projectId が不正です。	
タスク名は必須です。 projectId は必須です。 projectId が不正です。	
projectId は必須です。 不正なIDです。	Path必須:{id} (未指定は400)

DAO 詳細

DAO	メソッド
ProjectMemberDao	listByProject
ProjectMemberDao	listProjectIdsByUser
ProjectMemberDao	isMember
ProjectMemberDao	map
ProjectMemberDao	upsert
ProjectMemberDao	safeDecimal
DocumentDao	listByProject
DocumentDao	find
DocumentDao	insert
DocumentDao	delete
DocumentDao	map
DocumentDao	mapMeta
UserDao	list
UserDao	find
UserDao	findByLoginId
UserDao	insert
UserDao	update
UserDao	delete
UserDao	map
IssueDao	listByProject
IssueDao	find
IssueDao	insert
IssueDao	update
IssueDao	delete
IssueDao	map

ForumPostDao	listByThread
ForumPostDao	insert
ForumPostDao	map
ProjectFinanceDao	find
ProjectFinanceDao	upsert
ProjectFinanceDao	map
ProjectFinanceDao	safeDecimal
TaskDao	listByProject
TaskDao	insert
TaskDao	update
TaskDao	deleteByProjectExcept
TaskDao	setBaselineForProject
TaskDao	map
TaskDao	safeDecimal
ForumThreadDao	listByProject
ForumThreadDao	insert
ForumThreadDao	find
ForumThreadDao	map
AssignmentDao	listByProject
AssignmentDao	deleteByProject
AssignmentDao	insert
AssignmentDao	map
AssignmentDao	safeDecimal
ChatDao	listByProject
ChatDao	insert
ChatDao	map
ResourceDao	listByProject
ResourceDao	insert
ResourceDao	update
ResourceDao	deleteByProjectExcept
ResourceDao	map

ResourceDao	safeDecimal
WikiDao	listByProject
WikiDao	find
WikiDao	upsert
WikiDao	map
WorkflowDao	listByProject
WorkflowDao	find
WorkflowDao	insert
WorkflowDao	update
WorkflowDao	map
ProjectDao	list
ProjectDao	listByUser
ProjectDao	find
ProjectDao	insert
ProjectDao	update
ProjectDao	updateBaselineSetAt
ProjectDao	delete
ProjectDao	map
ProjectDao	safeDecimal
DependencyDao	listByProject
DependencyDao	deleteByProject
DependencyDao	insert
DependencyDao	map

役割	TX	備考
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
ProjectMemberDao: プロジェクト参加者のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
DocumentDao: ドキュメント情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
UserDao: ユーザー情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	
IssueDao: 不具合情報のDBアクセス。	DAO内取得	

ForumPostDao: フォーラム投稿のDBアクセス。	DAO内取得	
ForumPostDao: フォーラム投稿のDBアクセス。	DAO内取得	
ForumPostDao: フォーラム投稿のDBアクセス。	DAO内取得	
ProjectFinanceDao: プロジェクト予実のDBアクセス。	DAO内取得	
ProjectFinanceDao: プロジェクト予実のDBアクセス。	DAO内取得	
ProjectFinanceDao: プロジェクト予実のDBアクセス。	DAO内取得	
ProjectFinanceDao: プロジェクト予実のDBアクセス。	DAO内取得	
TaskDao: タスク情報のDBアクセス。	DAO内取得	
TaskDao: タスク情報のDBアクセス。	外部TX	
TaskDao: タスク情報のDBアクセス。	外部TX	
TaskDao: タスク情報のDBアクセス。	外部TX	
TaskDao: タスク情報のDBアクセス。	外部TX	
TaskDao: タスク情報のDBアクセス。	DAO内取得	
TaskDao: タスク情報のDBアクセス。	DAO内取得	
ForumThreadDao: フォーラムスレッドのDBアクセス。	DAO内取得	
ForumThreadDao: フォーラムスレッドのDBアクセス。	DAO内取得	
ForumThreadDao: フォーラムスレッドのDBアクセス。	DAO内取得	
ForumThreadDao: フォーラムスレッドのDBアクセス。	DAO内取得	
AssignmentDao: タスク割当のDBアクセス。	DAO内取得	
AssignmentDao: タスク割当のDBアクセス。	外部TX	
AssignmentDao: タスク割当のDBアクセス。	外部TX	
AssignmentDao: タスク割当のDBアクセス。	DAO内取得	
AssignmentDao: タスク割当のDBアクセス。	DAO内取得	
ChatDao: チャット情報のDBアクセス。	DAO内取得	
ChatDao: チャット情報のDBアクセス。	DAO内取得	
ChatDao: チャット情報のDBアクセス。	DAO内取得	
ResourceDao: リソース情報のDBアクセス。	DAO内取得	
ResourceDao: リソース情報のDBアクセス。	外部TX	
ResourceDao: リソース情報のDBアクセス。	外部TX	
ResourceDao: リソース情報のDBアクセス。	外部TX	
ResourceDao: リソース情報のDBアクセス。	DAO内取得	

ResourceDao: リソース情報のDBアクセス。	DAO内取得	
WikiDao: Wikiページの DBアクセス。	DAO内取得	
WikiDao: Wikiページの DBアクセス。	DAO内取得	
WikiDao: Wikiページの DBアクセス。	DAO内取得	
WikiDao: Wikiページの DBアクセス。	DAO内取得	
WorkflowDao: 承認ワークフローのDBアクセス。	DAO内取得	
WorkflowDao: 承認ワークフローのDBアクセス。	DAO内取得	
WorkflowDao: 承認ワークフローのDBアクセス。	DAO内取得	
WorkflowDao: 承認ワークフローのDBアクセス。	DAO内取得	
WorkflowDao: 承認ワークフローのDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	外部TX	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
ProjectDao: プロジェクト情報のDBアクセス。	DAO内取得	
DependencyDao: 依存関係のDBアクセス。	DAO内取得	
DependencyDao: 依存関係のDBアクセス。	外部TX	
DependencyDao: 依存関係のDBアクセス。	外部TX	
DependencyDao: 依存関係のDBアクセス。	DAO内取得	

モデル詳細

モデル	フィールド	型
Team	id	Long
Team	name	String
ProjectFinance	projectId	Long
ProjectFinance	revenueActual	BigDecimal
ProjectFinance	costActual	BigDecimal
ProjectFinance	laborCostActual	BigDecimal
ForumThread	id	Long
ForumThread	projectId	Long
ForumThread	title	String
ForumThread	createdBy	Long
ForumThread	createdAt	LocalDateTime
DocumentFile	id	Long
DocumentFile	projectId	Long
DocumentFile	taskId	Long
DocumentFile	filename	String
DocumentFile	contentType	String
DocumentFile	data	byte[]
DocumentFile	uploadedBy	Long
DocumentFile	createdAt	LocalDateTime
Resource	id	Long
Resource	projectId	Long
Resource	name	String
Resource	role	String
Resource	dailyHours	BigDecimal
Resource	costRate	BigDecimal
ForumPost	id	Long
ForumPost	threadId	Long
ForumPost	body	String
ForumPost	createdBy	Long
ForumPost	createdAt	LocalDateTime
User	id	Long
User	loginId	String
User	name	String
User	email	String
User	passwordHash	String
User	role	String
User	active	boolean
TaskAssignment	id	Long
TaskAssignment	taskId	Long
TaskAssignment	resourceId	Long
TaskAssignment	allocationHours	BigDecimal
ProjectMember	projectId	Long
ProjectMember	userId	Long

ProjectMember	role	String
ProjectMember	hourlyCost	BigDecimal
ProjectMember	allocationHours	BigDecimal
Issue	id	Long
Issue	projectId	Long
Issue	title	String
Issue	description	String
Issue	status	String
Issue	priority	Integer
Issue	severity	String
Issue	reporterId	Long
Issue	assigneeId	Long
Issue	createdAt	LocalDateTime
Issue	updatedAt	LocalDateTime
Issue	resolvedAt	LocalDateTime
WikiPage	id	Long
WikiPage	projectId	Long
WikiPage	title	String
WikiPage	content	String
WikiPage	version	Integer
WikiPage	updatedBy	Long
WikiPage	updatedAt	LocalDateTime
Task	id	Long
Task	projectId	Long
Task	parentId	Long
Task	sortOrder	Integer
Task	name	String
Task	description	String
Task	status	String
Task	priority	Integer
Task	assignedUserId	Long
Task	taskType	String
Task	startDate	LocalDate
Task	endDate	LocalDate
Task	dueDate	LocalDate
Task	progress	Integer
Task	plannedHours	BigDecimal
Task	actualHours	BigDecimal
Task	baselineStartDate	LocalDate
Task	baselineEndDate	LocalDate
Task	baselinePlannedHours	BigDecimal
Task	milestone	boolean
Project	id	Long
Project	name	String
Project	code	String
Project	description	String

Project	startDate	LocalDate
Project	endDate	LocalDate
Project	status	String
Project	budgetRevenue	BigDecimal
Project	budgetCost	BigDecimal
Project	progress	Integer
Project	taskCount	Integer
Project	workdays	String
Project	dailyHours	BigDecimal
Project	baselineSetAt	LocalDateTime
WorkflowRequest	id	Long
WorkflowRequest	projectId	Long
WorkflowRequest	taskId	Long
WorkflowRequest	requesterId	Long
WorkflowRequest	approverId	Long
WorkflowRequest	status	String
WorkflowRequest	message	String
WorkflowRequest	createdAt	LocalDateTime
WorkflowRequest	updatedAt	LocalDateTime
ChatMessage	id	Long
ChatMessage	projectId	Long
ChatMessage	userId	Long
ChatMessage	message	String
ChatMessage	createdAt	LocalDateTime
Dependency	id	Long
Dependency	projectId	Long
Dependency	predecessorId	Long
Dependency	successorId	Long
Dependency	type	String
Dependency	lagDays	Integer

備考

DTO詳細

DTO	フィールド	型
UserRequest	id	Long
UserRequest	loginId	String
UserRequest	name	String
UserRequest	email	String
UserRequest	role	String
UserRequest	password	String
UserRequest	active	Boolean
UserSummary	id	Long
UserSummary	loginId	String
UserSummary	name	String
UserSummary	role	String
UserSummary	active	boolean
ProjectPlan	project	Project

備考

ユーティリティ

クラス	メソッド
DbUtil	getConnection
HashUtil	sha256
Config	get

役割
DbUtil: 共通ユーティリティ。
HashUtil: 文字列のハッシュ生成を行う。
Config: 共通ユーティリティ。

DB 詳細

テーブル	カラム	型	PK
projects	id	BIGINT	
projects	name	VARCHAR(200)	
projects	code	VARCHAR(50)	
projects	description	TEXT	
projects	start_date	DATE	
projects	end_date	DATE	
projects	status	VARCHAR(30)	
projects	budget_revenue	DECIMAL(12,2)	
projects	budget_cost	DECIMAL(12,2)	
projects	workdays	VARCHAR(20)	
projects	daily_hours	DECIMAL(4,2)	
projects	baseline_set_at	DATETIME	
projects	created_at	DATETIME	
projects	updated_at	DATETIME	
users	id	BIGINT	
users	login_id	VARCHAR(60)	
users	name	VARCHAR(100)	
users	email	VARCHAR(120)	
users	password_hash	VARCHAR(64)	
users	role	VARCHAR(30)	
users	active	TINYINT(1)	
users	created_at	DATETIME	
teams	id	BIGINT	
teams	name	VARCHAR(120)	
teams	created_at	DATETIME	
team_members	team_id	BIGINT	PK
team_members	user_id	BIGINT	PK
project_members	project_id	BIGINT	PK
project_members	user_id	BIGINT	PK
project_members	role	VARCHAR(30)	
project_members	hourly_cost	DECIMAL(10,2)	
project_members	allocation_hours	DECIMAL(8,2)	
tasks	id	BIGINT	
tasks	project_id	BIGINT	
tasks	parent_id	BIGINT	
tasks	sort_order	INT	
tasks	name	VARCHAR(255)	
tasks	description	TEXT	
tasks	status	VARCHAR(20)	
tasks	priority	INT	

tasks	assigned_user_id	BIGINT	
tasks	task_type	VARCHAR(20)	
tasks	start_date	DATE	
tasks	end_date	DATE	
tasks	due_date	DATE	
tasks	progress	INT	
tasks	planned_hours	DECIMAL(8,2)	
tasks	actual_hours	DECIMAL(8,2)	
tasks	baseline_start_date	DATE	
tasks	baseline_end_date	DATE	
tasks	baseline_planned_hours	DECIMAL(8,2)	
tasks	is_milestone	TINYINT(1)	
tasks	created_at	DATETIME	
tasks	updated_at	DATETIME	
task_dependencies	id	BIGINT	
task_dependencies	project_id	BIGINT	
task_dependencies	predecessor_id	BIGINT	
task_dependencies	successor_id	BIGINT	
task_dependencies	dep_type	VARCHAR(2)	
task_dependencies	lag_days	INT	
task_dependencies	created_at	DATETIME	
resources	id	BIGINT	
resources	project_id	BIGINT	
resources	name	VARCHAR(100)	
resources	role	VARCHAR(100)	
resources	daily_hours	DECIMAL(4,2)	
resources	cost_rate	DECIMAL(10,2)	
resources	created_at	DATETIME	
resources	updated_at	DATETIME	
task_assignments	id	BIGINT	
task_assignments	task_id	BIGINT	
task_assignments	resource_id	BIGINT	
task_assignments	allocation_hours	DECIMAL(8,2)	
task_assignments	created_at	DATETIME	
project_finance	project_id	BIGINT	
project_finance	revenue_actual	DECIMAL(12,2)	
project_finance	cost_actual	DECIMAL(12,2)	
project_finance	labor_cost_actual	DECIMAL(12,2)	
project_finance	updated_at	DATETIME	
forum_threads	id	BIGINT	
forum_threads	project_id	BIGINT	

forum_threads	title	VARCHAR(200)	
forum_threads	created_by	BIGINT	
forum_threads	created_at	DATETIME	
forum_posts	id	BIGINT	
forum_posts	thread_id	BIGINT	
forum_posts	body	TEXT	
forum_posts	created_by	BIGINT	
forum_posts	created_at	DATETIME	
chat_messages	id	BIGINT	
chat_messages	project_id	BIGINT	
chat_messages	user_id	BIGINT	
chat_messages	message	TEXT	
chat_messages	created_at	DATETIME	
documents	id	BIGINT	
documents	project_id	BIGINT	
documents	task_id	BIGINT	
documents	filename	VARCHAR(200)	
documents	content_type	VARCHAR(120)	
documents	data	LONGBLOB	
documents	uploaded_by	BIGINT	
documents	created_at	DATETIME	
issues	id	BIGINT	
issues	project_id	BIGINT	
issues	title	VARCHAR(200)	
issues	description	TEXT	
issues	status	VARCHAR(20)	
issues	priority	INT	
issues	severity	VARCHAR(20)	
issues	reporter_id	BIGINT	
issues	assignee_id	BIGINT	
issues	created_at	DATETIME	
issues	updated_at	DATETIME	
issues	resolved_at	DATETIME	
workflow_requests	id	BIGINT	
workflow_requests	project_id	BIGINT	
workflow_requests	task_id	BIGINT	
workflow_requests	requester_id	BIGINT	
workflow_requests	approver_id	BIGINT	
workflow_requests	status	VARCHAR(20)	
workflow_requests	message	TEXT	
workflow_requests	created_at	DATETIME	

workflow_requests	updated_at	DATETIME	
wiki_pages	id	BIGINT	
wiki_pages	project_id	BIGINT	
wiki_pages	title	VARCHAR(200)	
wiki_pages	content	MEDIUMTEXT	
wiki_pages	version	INT	
wiki_pages	updated_by	BIGINT	
wiki_pages	updated_at	DATETIME	

FK	UNIQUE	NOT NULL	DEFAULT
		NOT NULL	
			NULL
			NULL
			NULL
			NULL
			'ACTIVE'
		NOT NULL	0.00
		NOT NULL	0.00
		NOT NULL	'1
		NOT NULL	8.00
			NULL
		NOT NULL	CURRENT_TIMESTAMP
		NOT NULL	CURRENT_TIMESTAMP
	UNIQUE	NOT NULL	
		NOT NULL	
			NULL
		NOT NULL	
		NOT NULL	'MEMBER'
		NOT NULL	1
		NOT NULL	CURRENT_TIMESTAMP
		NOT NULL	
		NOT NULL	CURRENT_TIMESTAMP
teams(id)		NOT NULL	
users(id)		NOT NULL	
projects(id)		NOT NULL	
users(id)		NOT NULL	
		NOT NULL	'MEMBER'
		NOT NULL	0.00
		NOT NULL	0.00
projects(id)		NOT NULL	
tasks(id)			NULL
		NOT NULL	0
		NOT NULL	
			NULL
		NOT NULL	'TODO'
		NOT NULL	3

users(id)			NULL
		NOT NULL	'TASK'
			NULL
			NULL
			NULL
		NOT NULL	0
		NOT NULL	0.00
		NOT NULL	0.00
			NULL
			NULL
			NULL
		NOT NULL	0
		NOT NULL	CURRENT_TIMESTAMP
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
tasks(id)		NOT NULL	
tasks(id)		NOT NULL	
		NOT NULL	'FS'
		NOT NULL	0
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
		NOT NULL	
			NULL
		NOT NULL	8.00
		NOT NULL	0.00
		NOT NULL	CURRENT_TIMESTAMP
		NOT NULL	CURRENT_TIMESTAMP
tasks(id)		NOT NULL	
resources(id)		NOT NULL	
		NOT NULL	0.00
		NOT NULL	CURRENT_TIMESTAMP
projects(id)			
		NOT NULL	0.00
		NOT NULL	0.00
		NOT NULL	0.00
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	

		NOT NULL	
users(id)			NULL
		NOT NULL	CURRENT_TIMESTAMP
forum_threads(id)		NOT NULL	
		NOT NULL	
users(id)			NULL
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
users(id)			NULL
		NOT NULL	
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
tasks(id)			NULL
		NOT NULL	
		NOT NULL	
		NOT NULL	
users(id)			NULL
		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
		NOT NULL	
			NULL
		NOT NULL	'OPEN'
		NOT NULL	3
		NOT NULL	'MEDIUM'
users(id)			NULL
users(id)			NULL
		NOT NULL	CURRENT_TIMESTAMP
		NOT NULL	CURRENT_TIMESTAMP
			NULL
projects(id)		NOT NULL	
tasks(id)			NULL
users(id)			NULL
users(id)			NULL
		NOT NULL	'PENDING'
			NULL
		NOT NULL	CURRENT_TIMESTAMP

		NOT NULL	CURRENT_TIMESTAMP
projects(id)		NOT NULL	
		NOT NULL	
		NOT NULL	
		NOT NULL	1
users(id)			NULL
		NOT NULL	CURRENT_TIMESTAMP

認証・認可

対象
全体
ADMIN
MANAGER
PROJECT_MEMBER

ルール
ログイン必須 (AuthFilter) - /login,/logout,/static**除外**
管理者権限が必要 (AccessControl.requireAdmin or SessionKeys.ROLE)
管理権限が必要 (AccessControl.requireManager)
プロジェクト参加者のみ (AccessControl.requireProjectMember)

備考
APIは未認証時401、Webは/loginへリダイレクト
例: /api/projects DELETE, /admin/users, /api/users
例: /api/projects POST/PUT
例: /api/plan, /api/tasks 等

エラーハンドリング

対象	例外/条件
API	SecurityException
API	IllegalArgumentException
API	**その他例外**
Web	**未認証**
Web	**ログインエラー**

レスポンス
403 FORBIDDEN + ApiErrorResponse
400 BAD_REQUEST + ApiErrorResponse
500 INTERNAL_SERVER_ERROR + ApiErrorResponse
AuthFilterが/loginへリダイレクト
LoginServletがエラーメッセージ表示

ビルド/実行

項目	内容
ビルド	mvn -q -DskipTests package
成果物	target/project-kanri.war
配備	Servlet 5 (Tomcat 10+) **にデプロイ**
設定	src/main/resources/app.properties (Config**経由で読込**)

Servlet 処理フロー（要約）

クラス	HTTP	Path	TX
IndexServlet	GET	/	N/A
DocsServlet	GET	/docs	N/A
LogoutServlet	GET	/logout	N/A
ProjectsServlet	GET	/projects	N/A
CalendarServlet	GET	/calendar	N/A
ReportsServlet	GET	/reports	N/A
FinanceServlet	GET	/finance	N/A
IssuesServlet	GET	/issues	N/A
WikiServlet	GET	/wiki	N/A
LoginServlet	GET	/login	N/A
LoginServlet	POST	/login	N/A
ProjectServlet	GET	/project	N/A
WorkflowServlet	GET	/workflow	N/A
KanbanServlet	GET	/kanban	N/A
AdminUsersServlet	GET	/admin/users	N/A
ForumServlet	GET	/forum	N/A
ApiChatServlet	GET	/api/chat	N/A
ApiChatServlet	POST	/api/chat	N/A

ApiProjectsServlet	GET	/api/projects/*	N/A
ApiProjectsServlet	POST	/api/projects/*	N/A
ApiProjectsServlet	PUT	/api/projects/*	N/A
ApiProjectsServlet	DELETE	/api/projects/*	N/A
ApiUsersServlet	GET	/api/users/*	N/A
ApiUsersServlet	POST	/api/users/*	N/A
ApiUsersServlet	PUT	/api/users/*	N/A

ApiUsersServlet	DELETE	/api/users/*	N/A
ApiReportsServlet	GET	/api/reports	N/A
ApiForumServlet	GET	/api/forum/*	N/A
ApiForumServlet	POST	/api/forum/*	N/A
ApiFinanceServlet	GET	/api/finance	N/A
ApiFinanceServlet	PUT	/api/finance	N/A
ApiAlertsServlet	GET	/api/alerts	N/A

ApiIssuesServlet	GET	/api/issues/*	N/A
ApiIssuesServlet	POST	/api/issues/*	N/A
ApiIssuesServlet	PUT	/api/issues/*	N/A
ApiIssuesServlet	DELETE	/api/issues/*	N/A
ApiWikiServlet	GET	/api/wiki/*	N/A
ApiWikiServlet	POST	/api/wiki/*	N/A
ApiWorkflowServlet	GET	/api/workflow/*	N/A

ApiWorkflowServlet	POST	/api/workflow/*	N/A
ApiWorkflowServlet	PUT	/api/workflow/*	N/A
ApiPlanServlet	GET	/api/plan	N/A
ApiPlanServlet	PUT	/api/plan	**明示**TX (commit/rollback)
ApiBaselineServlet	POST	/api/plan/baseline	**明示**TX (commit/rollback)
ApiDocumentsServlet	GET	/api/docs	N/A
ApiDocumentsServlet	POST	/api/docs	N/A

ApiDocumentsServlet	DELETE	/api/docs	N/A
ApiTasksServlet	GET	/api/tasks/*	N/A
ApiTasksServlet	POST	/api/tasks/*	N/A
ApiTasksServlet	PUT	/api/tasks/*	N/A

処理フロー
・認証: AuthFilterでログイン必須 ・リダイレクト
・認証: AuthFilterでログイン必須 ・JSP forward: docs.jsp
・リダイレクト
・認証: AuthFilterでログイン必須 ・JSP forward: projects.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: calendar.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: reports.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: finance.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: issues.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: wiki.jsp
・JSP forward: login.jsp
・入力取得: loginId, password ・バリデーション: ログインIDを入力してください。 / ユーザーが存在しません。 / パスワードが正しくありません。 ・DAO処理: userDao.findByLoginId ・リダイレクト
・認証: AuthFilterでログイン必須 ・JSP forward: project.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: workflow.jsp
・認証: AuthFilterでログイン必須 ・JSP forward: kanban.jsp
・認証: AuthFilterでログイン必須 ・認可: ADMIN ・JSP forward: admin_users.jsp ・sendError: 403
・認証: AuthFilterでログイン必須 ・JSP forward: forum.jsp
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: chatDao.listByProject ・JSON応答: 200
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: ChatMessage ・バリデーション: メッセージは必須です。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: chatDao.insert

・認証: AuthFilterでログイン必須 ・認可: LOGIN, PROJECT_MEMBER ・Path取得: {id} ・バリデーション: 不正なIDです。 ・DAO処理: projectDao.find, projectDao.list, projectDao.listByUser ・JSON応答: 200, 200, 200 ・sendError: 404 ・分岐/補足: ADMIN: 全件 / 非ADMIN: 所属プロジェクトのみ / Path未指定:一覧 / Path指定:{id}:詳細
・認証: AuthFilterでログイン必須 ・認可: MANAGER ・JSON読込: Project ・バリデーション: プロジェクト名は必須です。 ・DAO処理: projectDao.find, projectDao.insert, projectMemberDao.upsert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: MANAGER, PROJECT_MEMBER ・Path取得: {id} ・JSON読込: Project ・バリデーション: プロジェクト名は必須です。 / 不正なIDです。 ・DAO処理: projectDao.find, projectDao.update ・JSON応答: 200 ・sendError: 400 ・分岐/補足: Path必須:{id} (未指定は400)
・認証: AuthFilterでログイン必須 ・認可: ADMIN ・Path取得: {id} ・バリデーション: 不正なIDです。 ・DAO処理: projectDao.delete ・sendError: 400 ・分岐/補足: Path必須:{id} (未指定は400)
・認証: AuthFilterでログイン必須 ・DAO処理: userDao.list ・JSON応答: 200, 200 ・分岐/補足: ADMIN: 全件(User) / 非ADMIN: 有効ユーザーのみ(UserSummary)
・認証: AuthFilterでログイン必須 ・認可: ADMIN ・JSON読込: UserRequest ・バリデーション: ログインIDは必須です。 / 氏名は必須です。 / パスワードは必須です。 ・DAO処理: userDao.find, userDao.insert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: ADMIN ・Path取得: {id} ・JSON読込: UserRequest ・バリデーション: 不正なIDです。 ・DAO処理: userDao.find, userDao.update ・JSON応答: 200 ・sendError: 400, 404 ・分岐/補足: Path必須:{id} (未指定は400)

・認証: AuthFilterでログイン必須
・認可: ADMIN
・Path取得: {id}
・バリデーション: 不正なIDです。
・DAO処理: userDao.delete
・sendError: 400
・分岐/補足: Path必須:{id} (未指定は400)

・認証: AuthFilterでログイン必須
・認可: PROJECT_MEMBER
・入力取得: projectId
・バリデーション: projectId は必須です。 / projectId が不正です。
・DAO処理: financeDao.find, projectDao.find, taskDao.listByProject
・JSON応答: 200

・認証: AuthFilterでログイン必須
・認可: PROJECT_MEMBER
・入力取得: projectId, threadId
・DAO処理: postDao.listByThread, threadDao.find, threadDao.listByProject
・JSON応答: 200, 200
・sendError: 404
・分岐/補足: Path=/posts: threadId必須(投稿) / Path未指定: projectId必須(スレッド) / /posts: 200 posts / base: 200 threads

・認証: AuthFilterでログイン必須
・認可: PROJECT_MEMBER
・入力取得: projectId, threadId
・JSON読込: ForumPost, ForumThread
・DAO処理: postDao.insert, threadDao.find, threadDao.insert
・JSON応答: 201
・sendError: 404
・分岐/補足: Path=/posts: threadId必須(投稿) / Path未指定: projectId必須(スレッド) / /posts: 201のみ(Bodyなし)

・認証: AuthFilterでログイン必須
・認可: PROJECT_MEMBER
・入力取得: projectId
・バリデーション: projectId は必須です。 / projectId が不正です。
・DAO処理: financeDao.find, projectDao.find
・JSON応答: 200

・認証: AuthFilterでログイン必須
・認可: MANAGER, PROJECT_MEMBER
・入力取得: projectId
・JSON読込: Map
・バリデーション: projectId は必須です。 / projectId が不正です。
・DAO処理: financeDao.upsert, projectDao.find, projectDao.update
・sendError: 404

・認証: AuthFilterでログイン必須
・認可: PROJECT_MEMBER
・入力取得: projectId
・バリデーション: projectId は必須です。 / projectId が不正です。
・DAO処理: projectDao.find, taskDao.listByProject
・JSON応答: 200

・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: issueDao.listByProject ・JSON応答: 200
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: Issue ・バリデーション: タイトルは必須です。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: issueDao.insert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・Path取得: {id} ・JSON読込: Issue ・バリデーション: 不正なIDです。 ・DAO処理: issueDao.find, issueDao.update ・JSON応答: 200 ・sendError: 400, 404 ・分岐/補足: Path必須:{id} (未指定は400)
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・Path取得: {id} ・バリデーション: 不正なIDです。 ・DAO処理: issueDao.delete, issueDao.find ・sendError: 400, 404 ・分岐/補足: Path必須:{id} (未指定は400)
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: id, projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: wikiDao.find, wikiDao.listByProject ・JSON応答: 200, 200 ・sendError: 404
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: WikiPage ・バリデーション: タイトルは必須です。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: wikiDao.upsert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: workflowDao.listByProject ・JSON応答: 200

・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: WorkflowRequest ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: workflowDao.insert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・Path取得: {id} ・JSON読込: WorkflowRequest ・バリデーション: 不正なIDです。 ・DAO処理: workflowDao.find, workflowDao.update ・JSON応答: 200 ・sendError: 400, 404 ・分岐/補足: Path必須:{id} (未指定は400)
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: assignmentDao.listByProject, dependencyDao.listByProject, projectDao.find, resourceDao.listByProject, taskDao.listByProject ・JSON応答: 200 ・sendError: 404
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: ProjectPlan ・バリデーション: 計画データがありません。 / プロジェクト名は必須です。 / リソース名は必須です。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: assignmentDao.deleteByProject, assignmentDao.insert, dependencyDao.deleteByProject, dependencyDao.insert, projectDao.update, resourceDao.deleteByProjectExcept, resourceDao.insert, resourceDao.update taskDao.deleteByProjectExcept, taskDao.insert, taskDao.update
・認証: AuthFilterでログイン必須 ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: projectDao.updateBaselineSetAt, taskDao.setBaselineForProject
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: id, projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: documentDao.find, documentDao.listByProject ・JSON応答: 200 ・ファイル応答: Content-Disposition/Content-Type ・sendError: 404 ・分岐/補足: id指定時: ファイルDL (Content-Disposition/Content-Type) / projectId指定時: JSON一覧
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId, taskId ・ファイル取得: multipart(file) ・バリデーション: ファイルがありません。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: documentDao.insert ・JSON応答: 201

・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: id ・DAO処理: documentDao.delete, documentDao.find ・sendError: 400, 404
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・バリデーション: projectId は必須です。 / projectId が不正です。 ・DAO処理: taskDao.listByProject ・JSON応答: 200
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・入力取得: projectId ・JSON読込: Task ・バリデーション: タスク名は必須です。 / projectId は必須です。 / projectId が不正です。 ・DAO処理: taskDao.insert ・JSON応答: 201
・認証: AuthFilterでログイン必須 ・認可: PROJECT_MEMBER ・Path取得: {id} ・JSON読込: Task ・バリデーション: projectId は必須です。 / 不正なIDです。 ・DAO処理: taskDao.update ・JSON応答: 200 ・sendError: 400 ・分岐/補足: Path必須:{id} (未指定は400)

DAO SQL 詳細

DAO	メソッド
ProjectMemberDao	listByProject
ProjectMemberDao	listProjectIdsByUser
ProjectMemberDao	isMember
ProjectMemberDao	upsert
DocumentDao	listByProject
DocumentDao	find
DocumentDao	insert
DocumentDao	delete
UserDao	list
UserDao	find
UserDao	findByLoginId
UserDao	insert
UserDao	update
UserDao	delete
IssueDao	listByProject
IssueDao	find
IssueDao	insert

IssueDao	update
IssueDao	delete
ForumPostDao	listByThread
ForumPostDao	insert
ProjectFinanceDao	find
ProjectFinanceDao	upsert
TaskDao	listByProject
TaskDao	insert
TaskDao	update

TaskDao	deleteByProjectExcept
TaskDao	deleteByProjectExcept
TaskDao	setBaselineForProject
ForumThreadDao	listByProject
ForumThreadDao	insert
ForumThreadDao	find
AssignmentDao	listByProject
AssignmentDao	deleteByProject
AssignmentDao	insert
ChatDao	listByProject
ChatDao	insert
ResourceDao	listByProject
ResourceDao	insert
ResourceDao	update
ResourceDao	deleteByProjectExcept
ResourceDao	deleteByProjectExcept
WikiDao	listByProject
WikiDao	find
WikiDao	upsert
WikiDao	upsert
WorkflowDao	listByProject
WorkflowDao	find
WorkflowDao	insert

WorkflowDao	update
ProjectDao	list
ProjectDao	listByUser
ProjectDao	find
ProjectDao	insert
ProjectDao	update
ProjectDao	updateBaselineSetAt
ProjectDao	delete
DependencyDao	listByProject
DependencyDao	deleteByProject
DependencyDao	insert

SQL
SELECT * FROM project_members WHERE project_id = ?
SELECT project_id FROM project_members WHERE user_id = ?
SELECT 1 FROM project_members WHERE project_id = ? AND user_id = ? LIMIT 1
INSERT INTO project_members (project_id, user_id, role, hourly_cost, allocation_hours) VALUES (?, ?, ?, ?, ?) ON DUPLICATE KEY UPDATE role=VALUES(role), hourly_cost=VALUES(hourly_cost), allocation_hours=VALUES(allocation_hours)
SELECT id, project_id, task_id, filename, content_type, uploaded_by, created_at FROM documents WHERE project_id = ? ORDER BY id DESC
SELECT * FROM documents WHERE id = ?
INSERT INTO documents (project_id, task_id, filename, content_type, data, uploaded_by) VALUES (?, ?, ?, ?, ?, ?)
DELETE FROM documents WHERE id = ?
SELECT * FROM users ORDER BY id
SELECT * FROM users WHERE id = ?
SELECT * FROM users WHERE login_id = ?
INSERT INTO users (login_id, name, email, password_hash, role, active) VALUES (?, ?, ?, ?, ?, ?)
UPDATE users SET login_id=?, name=?, email=?, password_hash=?, role=?, active=? WHERE id=?
DELETE FROM users WHERE id = ?
SELECT * FROM issues WHERE project_id = ? ORDER BY id DESC
SELECT * FROM issues WHERE id = ?
INSERT INTO issues (project_id, title, description, status, priority, severity, reporter_id, assignee_id) VALUES (?, ?, ?, ?, ?, ?, ?, ?)

UPDATE issues SET title=?, description=?, status=?, priority=?, severity=?, reporter_id=?, assignee_id=?, resolved_at=? WHERE id=?
DELETE FROM issues WHERE id = ?
SELECT * FROM forum_posts WHERE thread_id = ? ORDER BY id
INSERT INTO forum_posts (thread_id, body, created_by) VALUES (?, ?, ?)
SELECT * FROM project_finance WHERE project_id = ?
INSERT INTO project_finance (project_id, revenue_actual, cost_actual, labor_cost_actual) VALUES (?, ?, ?, ?) ON DUPLICATE KEY UPDATE revenue_actual=VALUES(revenue_actual), cost_actual=VALUES(cost_actual), labor_cost_actual=VALUES(labor_cost_actual)
SELECT * FROM tasks WHERE project_id = ? ORDER BY COALESCE(parent_id, 0), sort_order
INSERT INTO tasks (project_id, parent_id, sort_order, name, description, status, priority, assigned_user_id, task_type, start_date, end_date, due_date, progress, planned_hours, actual_hours, baseline_start_date, baseline_end_date, baseline_planned_hours, is_milestone) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
UPDATE tasks SET parent_id=?, sort_order=?, name=?, description=?, status=?, priority=?, assigned_user_id=?, task_type=?, start_date=?, end_date=?, due_date=?, progress=?, planned_hours=?, actual_hours=?, is_milestone=? WHERE id=?

DELETE FROM tasks WHERE project_id = ?
DELETE FROM tasks WHERE project_id = ? AND id NOT IN (? ...)
UPDATE tasks SET baseline_start_date = start_date, baseline_end_date = end_date, baseline_planned_hours = planned_hours WHERE project_id = ?
SELECT * FROM forum_threads WHERE project_id = ? ORDER BY id DESC
INSERT INTO forum_threads (project_id, title, created_by) VALUES (?, ?, ?)
SELECT * FROM forum_threads WHERE id = ?
SELECT ta.* FROM task_assignments ta INNER JOIN tasks t ON t.id = ta.task_id WHERE t.project_id = ?
DELETE ta FROM task_assignments ta INNER JOIN tasks t ON t.id = ta.task_id WHERE t.project_id = ?
INSERT INTO task_assignments (task_id, resource_id, allocation_hours) VALUES (?, ?, ?)
SELECT * FROM chat_messages WHERE project_id = ? ORDER BY id DESC LIMIT ?
INSERT INTO chat_messages (project_id, user_id, message) VALUES (?, ?, ?)
SELECT * FROM resources WHERE project_id = ? ORDER BY id
INSERT INTO resources (project_id, name, role, daily_hours, cost_rate) VALUES (?, ?, ?, ?, ?)
UPDATE resources SET name=?, role=?, daily_hours=?, cost_rate=? WHERE id=? AND project_id=?
DELETE FROM resources WHERE project_id = ?
DELETE FROM resources WHERE project_id = ? AND id NOT IN (? ...)
SELECT * FROM wiki_pages WHERE project_id = ? ORDER BY updated_at DESC
SELECT * FROM wiki_pages WHERE id = ?
INSERT INTO wiki_pages (project_id, title, content, version, updated_by) VALUES (?, ?, ?, ?, ?)
UPDATE wiki_pages SET title=?, content=?, version=?, updated_by=? WHERE id=?
SELECT * FROM workflow_requests WHERE project_id = ? ORDER BY id DESC
SELECT * FROM workflow_requests WHERE id = ?
INSERT INTO workflow_requests (project_id, task_id, requester_id, approver_id, status, message) VALUES (?, ?, ?, ?, ?, ?)

UPDATE workflow_requests SET task_id=?, requester_id=?, approver_id=?, status=?, message=? WHERE id=?
SELECT p.*, COALESCE(ROUND(AVG(t.progress)),0) AS progress, COUNT(t.id) AS task_count FROM projects p LEFT JOIN tasks t ON t.project_id = p.id GROUP BY p.id ORDER BY p.id DESC
SELECT p.*, COALESCE(ROUND(AVG(t.progress)),0) AS progress, COUNT(t.id) AS task_count FROM projects p JOIN project_members pm ON pm.project_id = p.id LEFT JOIN tasks t ON t.project_id = p.id WHERE pm.user_id = ? GROUP BY p.id ORDER BY p.id DESC
SELECT * FROM projects WHERE id = ?
INSERT INTO projects (name, code, description, start_date, end_date, status, budget_revenue, budget_cost, workdays, daily_hours) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
UPDATE projects SET name=?, code=?, description=?, start_date=?, end_date=?, status=?, budget_revenue=?, budget_cost=?, workdays=?, daily_hours=? WHERE id=?
UPDATE projects SET baseline_set_at = NOW() WHERE id = ?
DELETE FROM projects WHERE id = ?
SELECT * FROM task_dependencies WHERE project_id = ?
DELETE FROM task_dependencies WHERE project_id = ?
INSERT INTO task_dependencies (project_id, predecessor_id, successor_id, dep_type, lag_days) VALUES (?, ?, ?, ?, ?)

Params	Tables
1:projectId	project_members
1:userId	project_members
1:projectId 2:userId	project_members
1:m.getProjectId(2:m.getUserId(3:m.getRole(4:safeDecimal(m.getHourlyCost(5:safeDecimal(m.getAllocationHours (	project_members, role
1:projectId	documents
1:id	documents
1:d.getProjectId(2:d.getTaskId(3:d.getFilename(4:d.getContentType(5:d.getData(6:d.getUploadedBy(	documents
1:id	documents
	users
1:id	users
1:loginId	users
1:u.getLoginId(2:u.getName(3:u.getEmail(4:u.getPasswordHash(5:u.getRole(6:u.isActive(	users
1:u.getLoginId(2:u.getName(3:u.getEmail(4:u.getPasswordHash(5:u.getRole(6:u.isActive(7:u.getId(	users
1:id	users
1:projectId	issues
1:id	issues
1:i.getProjectId(2:i.getTitle(3:i.getDescription(4:i.getStatus(5:i.getPriority(6:i.getSeverity(7:i.getReporterId(8:i.getAssigneeId(	issues

1:i.getTitle(2:i.getDescription(3:i.getStatus(4:i.getPriority(5:i.getSeverity(6:i.getReporterId(7:i.getAssigneeId(8:Timestamp.valueOf(i.getResolvedAt(9:i.getId(	issues
1:id	issues
1:threadId	forum_posts
1:p.getThreadId(2:p.getBody(3:p.getCreatedBy(	forum_posts
1:projectId	project_finance
1:f.getProjectId(2:safeDecimal(f.getRevenueActual(3:safeDecimal(f.getCostActual(4:safeDecimal(f.getLaborCostActual(	project_finance, revenue_actual
1:projectId	tasks
1:t.getProjectId(2:t.getParentId(3:t.getSortOrder(4:t.getName(5:t.getDescription(6:t.getStatus(7:t.getPriority(8:t.getAssignedUserId(9:t.getTaskType(10:(t.getStartDate(11:(t.getEndDate(12:(t.getDueDate(13:t.getProgress(14:safeDecimal(t.getPlannedHours(15:safeDecimal(t.getActualHours(16:Types.DATE 17:Types.DATE 18:Types.DECIMAL 19:t.isMilestone(	tasks
1:t.getParentId(2:t.getSortOrder(3:t.getName(4:t.getDescription(5:t.getStatus(6:t.getPriority(7:t.getAssignedUserId(8:t.getTaskType(9:(t.getStartDate(10:(t.getEndDate(11:(t.getDueDate(12:t.getProgress(13:safeDecimal(t.getPlannedHours(14:safeDecimal(t.getActualHours(15:t.isMilestone(16:t.getId(	tasks

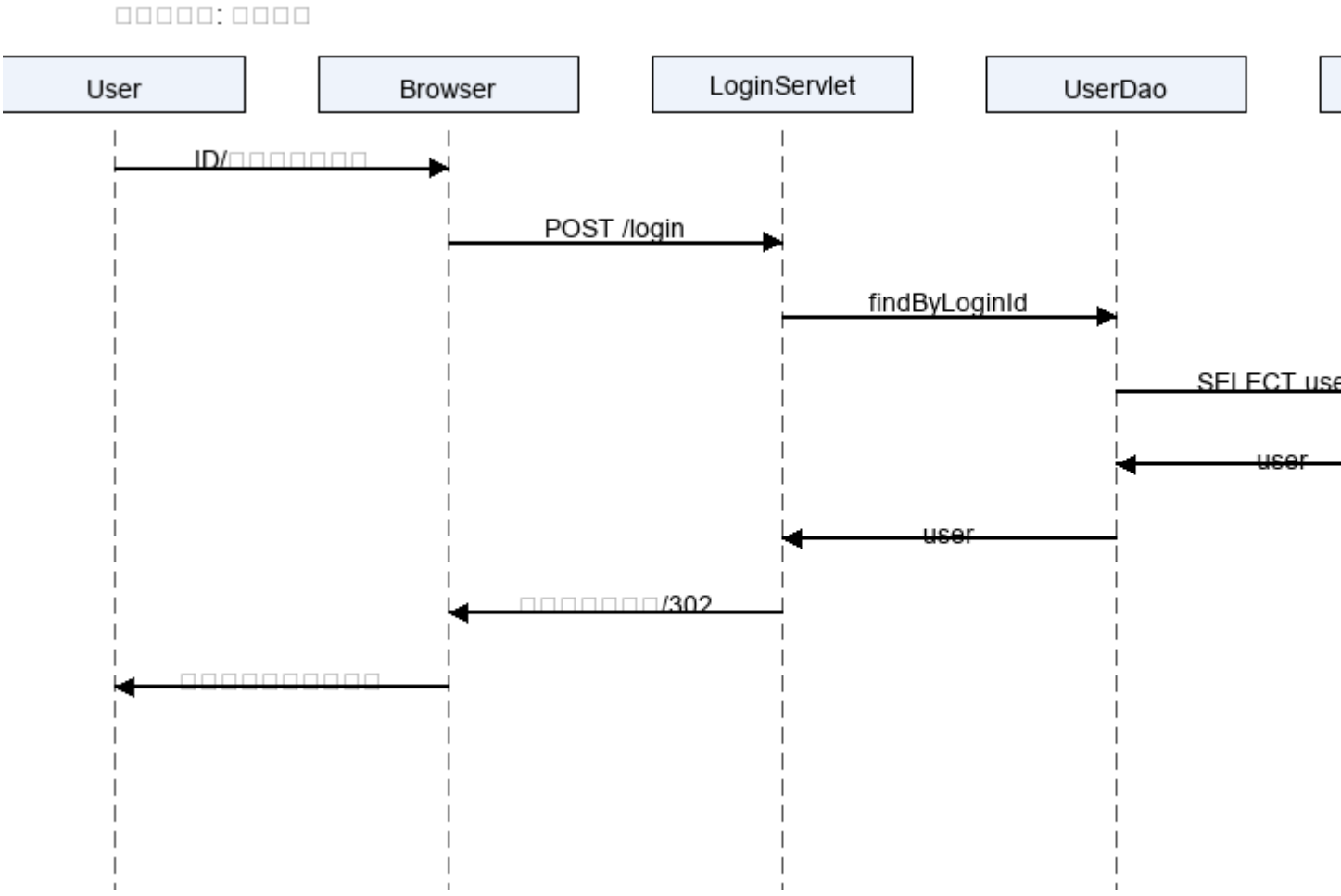
1:projectId	tasks
1:projectId	tasks
1:projectId	tasks
1:projectId	forum_threads
1:t.getProjectId(2:t.getTitle(3:t.getCreatedBy(	forum_threads
1:id	forum_threads
1:projectId	task_assignments, tasks
1:projectId	task_assignments, tasks
1:a.getTaskId(2:a.getResourceId(3:safeDecimal(a.getAllocationHours(	task_assignments
1:projectId 2:limit	chat_messages
1:m.getProjectId(2:m.getUserId(3:m.getMessage(	chat_messages
1:projectId	resources
1:r.getProjectId(2:r.getName(3:r.getRole(4:safeDecimal(r.getDailyHours(5:safeDecimal(r.getCostRate(	resources
1:r.getName(2:r.getRole(3:safeDecimal(r.getDailyHours(4:safeDecimal(r.getCostRate(5:r.getId(6:r.getProjectId(	resources
1:projectId	resources
1:projectId	resources
1:projectId	wiki_pages
1:id	wiki_pages
1:p.getProjectId(2:p.getTitle(3:p.getContent(4:p.getVersion(5:p.getUpdatedBy(	wiki_pages
1:p.getProjectId(2:p.getTitle(3:p.getContent(4:p.getVersion(5:p.getUpdatedBy(	wiki_pages
1:projectId	workflow_requests
1:id	workflow_requests
1:r.getProjectId(2:r.getTaskId(3:r.getRequesterId(4:r.getApproverId(5:r.getStatus(6:r.getMessage(	workflow_requests

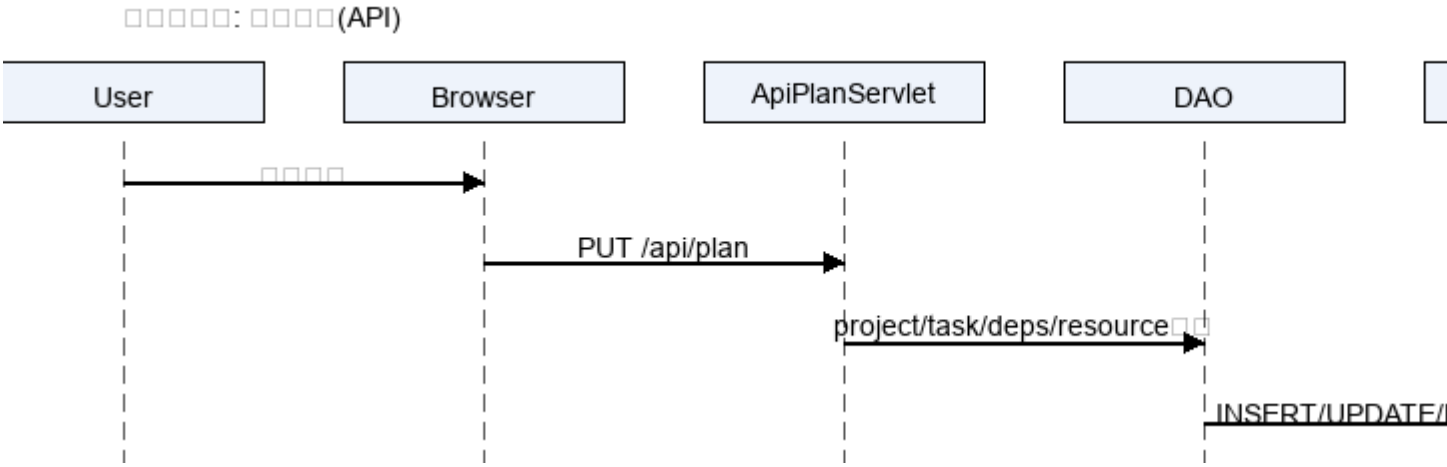
1:r.getTaskId(2:r.getRequesterId(3:r.getApproverId(4:r.getStatus(5:r.getMessage(6:r.getId(	workflow_requests
	projects, tasks
1:userId	project_members, projects, tasks
1:id	projects
1:p.getName(2:p.getCode(3:p.getDescription(4:(p.getStartDate(5:(p.getEndDate(6:p.getStatus(7:safeDecimal(p.getBudgetRevenue(8:safeDecimal(p.getBudgetCost(10:safeDecimal(p.getDailyHours(	projects
1:p.getName(2:p.getCode(3:p.getDescription(4:(p.getStartDate(5:(p.getEndDate(6:p.getStatus(7:safeDecimal(p.getBudgetRevenue(8:safeDecimal(p.getBudgetCost(10:safeDecimal(p.getDailyHours(11:p.getId(	projects
1:projectId	projects
1:id	projects
1:projectId	task_dependencies
1:projectId	task_dependencies
1:d.getProjectId(2:d.getPredecessorId(3:d.getSuccessorId(4:d.getType(5:d.getLagDays(	task_dependencies

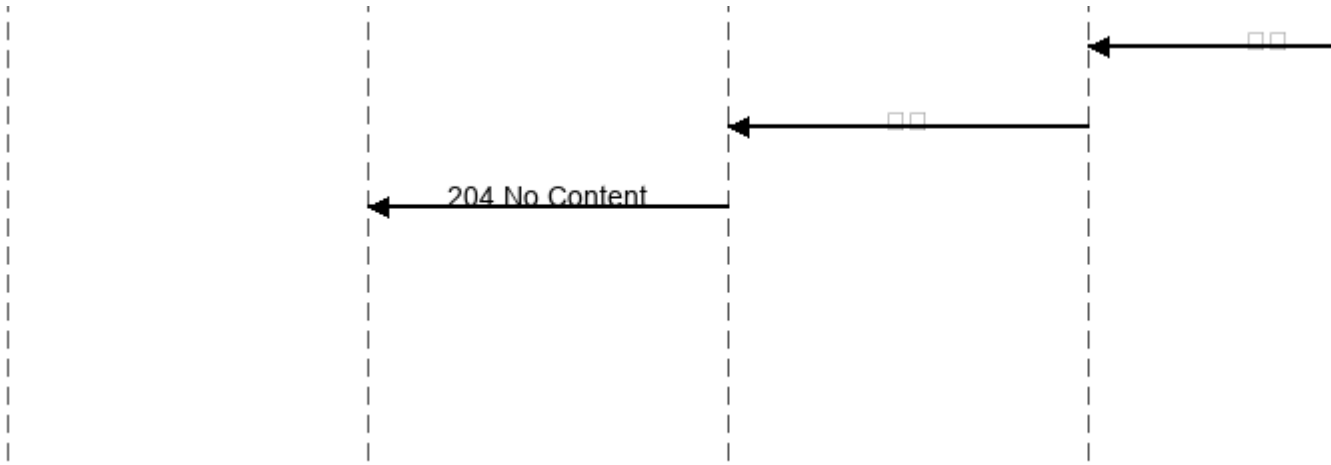
シーケンス図

上：図（画像）／下：PlantUML ソース

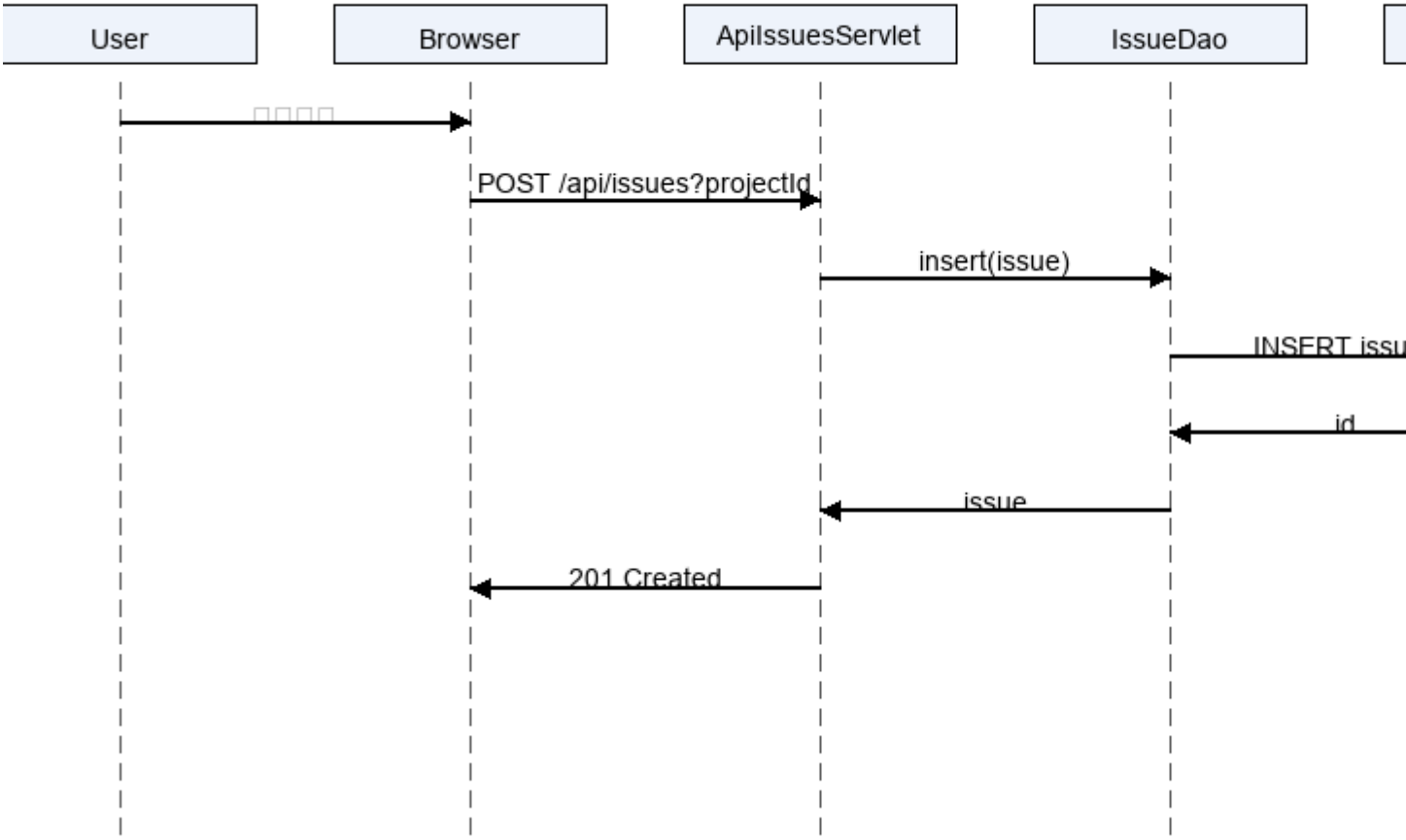
シーケンス: ログイン



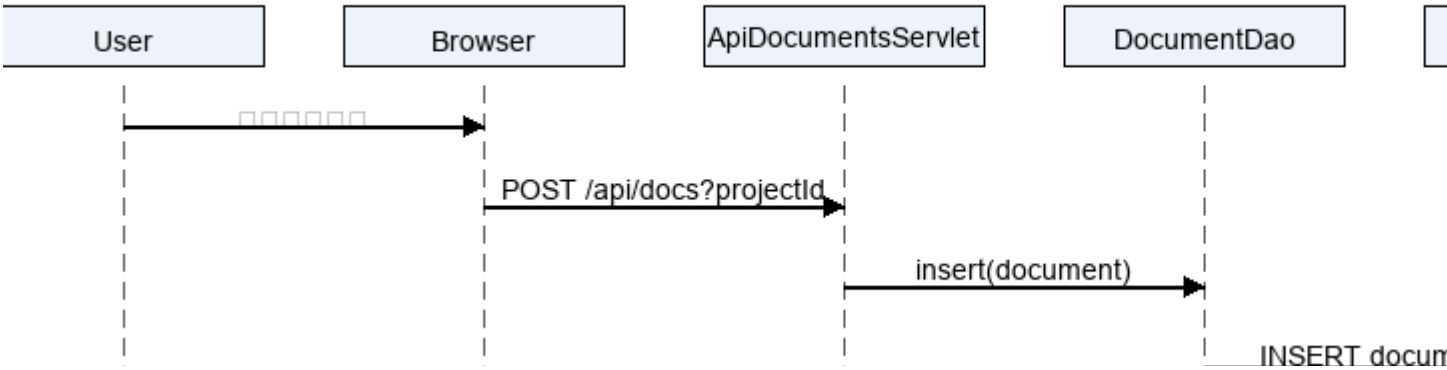


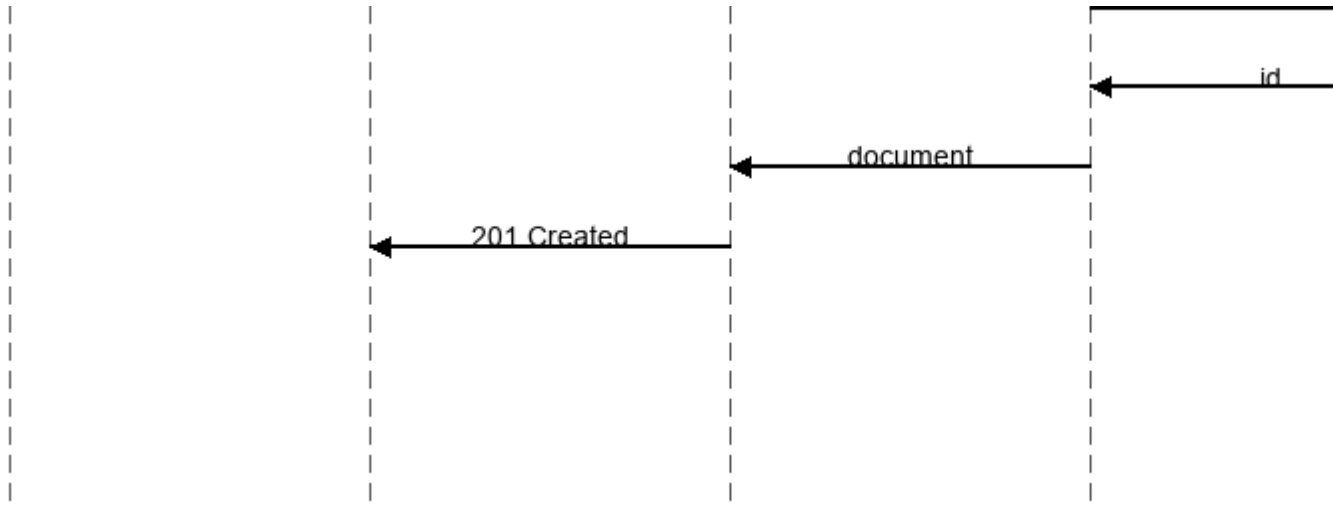

204 No Content

□□□□□: □□□□(API)

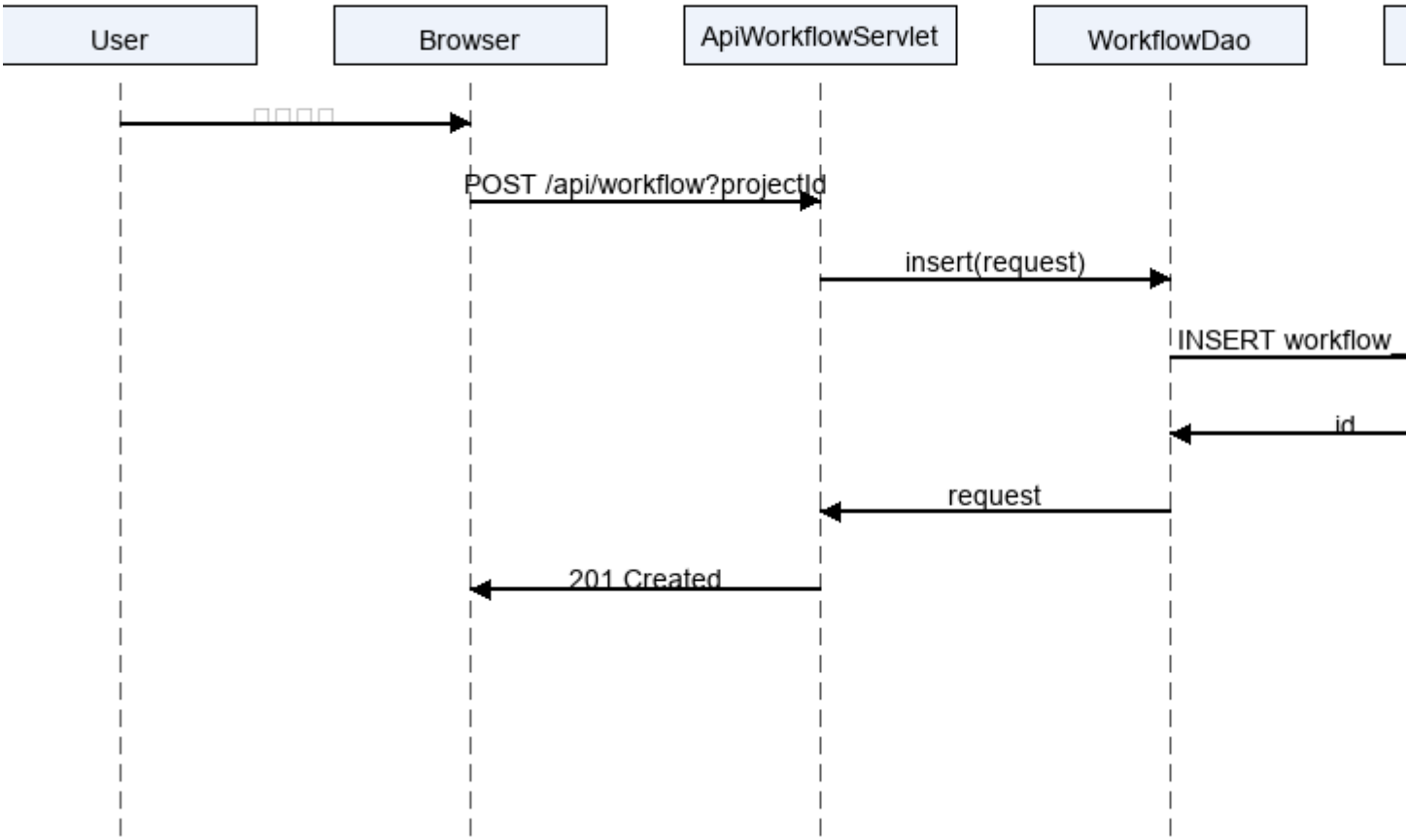

User
Browser
ApiIssuesServlet
IssueDao
POST /api/issues?projectId
insert(issue)
201 Created

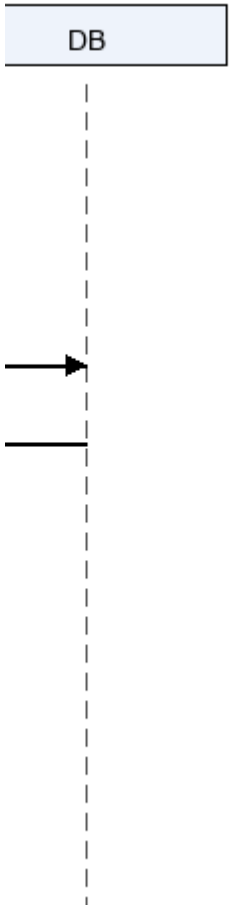
□□□□□: □□□□□□□□□(API)

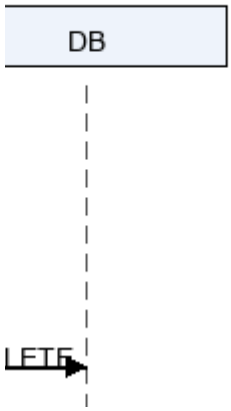

User
Browser
ApiDocumentsServlet
DocumentDao
POST /api/docs?projectId
insert(document)

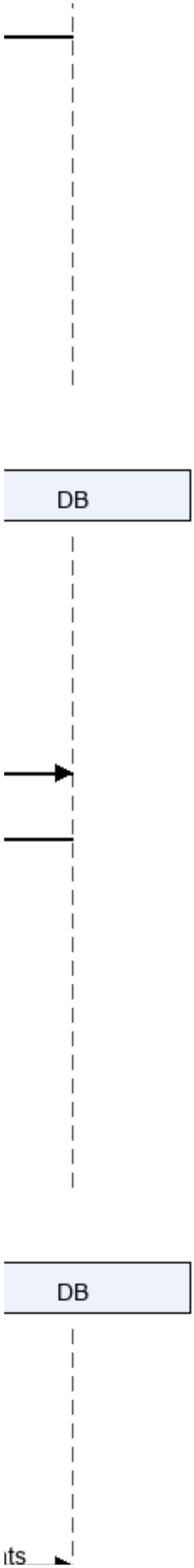

id
document
201 Created

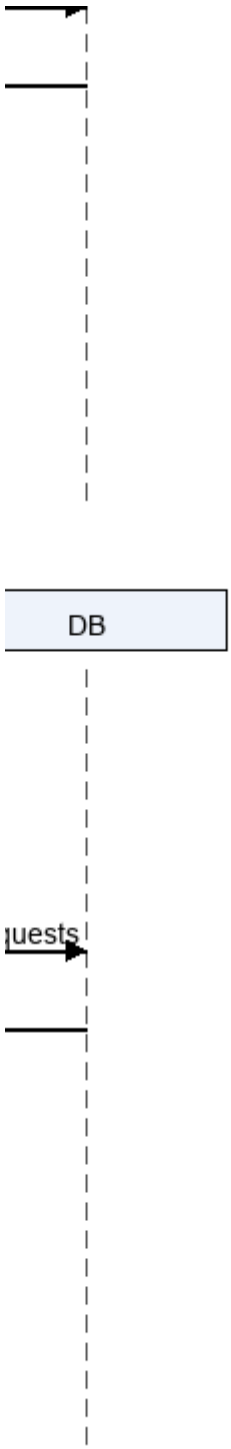
□□□□□: □□□□□□□□□(API)


User
Browser
ApiWorkflowServlet
WorkflowDao
POST /api/workflow?projectId
insert(request)
INSERT workflow
id
request
201 Created


DB


DB
LETE


DB
DB
ts


DB
uests